



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Tema 3.3: Capa de Controladores, APIs RESTful y Documentación con OpenAPI

Resumen

Esta lectura conceptual aborda el diseño e implementación de la **Capa de Controladores** en aplicaciones Spring Boot. Se profundiza en los fundamentos del protocolo **HTTP** (verbos, códigos de estado y encabezados) y en los principios arquitectónicos de **REST** en contraste con arquitecturas de microservicios y SOA. Finalmente, se detalla la configuración y el uso de **OpenAPI** / **Swagger** mediante la biblioteca Springdoc para generar documentación interactiva y estandarizada de las APIs expuestas al frontend.

Concepto Principal (El 'Qué')

1. La Capa de Controladores (@RestController)

En el diseño de arquitectura lógica multicapa, la capa de controladores (o capa de presentación de API) es el punto de entrada de la aplicación. Su función exclusiva es recibir solicitudes de red, validar formalmente la estructura de los datos entrantes (utilizando DTOs), delegar la ejecución a la capa de negocios y construir la respuesta de red correspondiente.

En Spring Boot, esta capa se implementa utilizando clases decoradas con la anotación `@RestController`. Esta anotación combina `@Controller` y `@ResponseBody`, indicando que la clase gestiona solicitudes HTTP y que los métodos retornan datos directamente serializados (típicamente en formato **JSON**) en el cuerpo de la respuesta, omitiendo la resolución de vistas HTML tradicionales.

2. Aspectos Fundamentales del Protocolo HTTP

HTTP (Hypertext Transfer Protocol) es un protocolo sin estado (*stateless*) basado en el esquema petición-respuesta. Para diseñar APIs RESTful profesionales, es obligatorio utilizar sus componentes de forma semántica:

A. Métodos HTTP (Verbos)

Declaran la acción que el cliente desea realizar sobre un recurso: * **GET**: Recupera un recurso o colección sin provocar efectos secundarios en el servidor (operación segura e idempotente). * **POST**: Crea un nuevo recurso. No es idempotente. * **PUT**: Reemplaza por completo un recurso existente. Es idempotente. * **DELETE**: Elimina un recurso específico. Es idempotente. * **PATCH**: Realiza modificaciones parciales a un recurso.

B. Códigos de Estado (Status Codes)

Son números de tres dígitos retornados por el servidor para informar el resultado de la solicitud: * **2xx** (Éxito): **200 OK** (Solicitud exitosa), **201 Created** (Recurso creado con éxito), **204 No Content** (Éxito sin datos de retorno). * **3xx** (Redirección): Indica que el cliente debe realizar acciones adicionales. * **4xx** (Error del Cliente): **400 Bad Request** (Datos inválidos o mal formados), **401 Unauthorized** (Falta autenticación), **403 Forbidden** (Autenticado pero sin permisos), **404 Not Found** (Recurso inexistente), **409 Conflict** (Violación de restricciones, como duplicidad). * **5xx** (Error del Servidor): **500 Internal Server Error** (Fallo inesperado del sistema), **503 Service Unavailable** (Servidor sobrecargado).

C. Encabezados HTTP (Headers)

Metadatos clave-valor transmitidos junto con la petición o respuesta. Algunos esenciales son `Content-Type` (indica el formato de los datos, ej. `application/json`) y `Authorization` (para transmitir tokens de acceso como JWT).

3. De SOA a Microservicios

- **SOA (Service-Oriented Architecture):** Patrón clásico que promueve la integración de aplicaciones mediante servicios reutilizables acoplados a través de un bus de servicios empresarial (ESB) y protocolos densos (ej. SOAP, XML).
- **Microservicios:** Evolución de SOA donde la aplicación se descompone en un conjunto de servicios autónomos, pequeños y desplegados de manera independiente. Cada servicio posee su propia base de datos y se comunican a través de protocolos ligeros como **APIs REST** sobre HTTP. Esto facilita el desacoplamiento físico entre el backend y los clientes front-end (Angular) o móviles.

4. Documentación Automática con OpenAPI / Swagger

Una API sin documentar es inútil para los desarrolladores de front-end o móviles. **OpenAPI** es una especificación estándar de la industria para describir APIs REST de forma independiente del lenguaje. **Swagger** es el conjunto de herramientas visuales que renderizan esta especificación.

Spring Boot facilita esto incorporando la dependencia `springdoc-openapi-starter-webmvc-ui`. En tiempo de compilación, el framework analiza el código y las anotaciones para generar un JSON de descripción y una interfaz web interactiva (`/swagger-ui/index.html`) que permite probar los endpoints directamente desde el navegador.

Analogía (La Intuición)

- **El Mesero del Restaurante (El Controlador):** Imagine que entra a una pizzería de la UCR. Usted no pasa directamente a la cocina a encender los hornos (base de datos) ni a amasar la harina (lógica de negocio). En su lugar, se sienta y se comunica exclusivamente con el mesero (Capa de Controladores). El mesero toma su orden (solicitud HTTP), verifica que la pizza que pide exista en el menú (validación de DTO), lleva el pedido a la cocina (capa de servicios) y regresa con el plato terminado. Si la pizza se preparó con éxito, se la entrega con una sonrisa (200 OK). Si usted pide sushi en una pizzería, el mesero le responde

directamente que no está en el menú (400 Bad Request), sin molestar a los cocineros.

- **La Carta en la Entrada (OpenAPI/Swagger):** La pizarra que se coloca fuera de la pizzería detallando qué ingredientes lleva cada plato, sus precios y cuáles bebidas están disponibles es la especificación de OpenAPI. Le permite a cualquiera entender exactamente qué puede pedir y cómo lo recibirá antes de interactuar con el mesero.

Ejemplo de Código e Implementación (El 'Cómo')

A continuación se muestra la implementación del controlador REST para gestionar cursos en la UCR. Incorpora anotaciones de OpenAPI para documentar las operaciones de consulta y creación de recursos, respetando el límite de longitud de líneas.

1. Dependencia Maven (`pom.xml`)

Incorpore la biblioteca Springdoc en las dependencias de su proyecto:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.5.0</version>
</dependency>
```

2. Objeto de Transferencia de Datos (`CursoDTO.java`)

Cree el DTO para el intercambio de datos en la ruta:

`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica2/dto/CursoDTO.java`

```
package cr.ac.ucr.paraiso.ie.carnet.practica2.dto;

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;

@Schema(description = "Modelo para la creación de un curso")
public class CursoDTO {
```

```

    @Schema(description = "Nombre del curso académico", example = "Sistemas")
    @NotBlank(message = "El nombre del curso no puede estar vacío.")
    private String nombre;

    @Schema(description = "Cupos máximos para estudiantes", example = "30")
    @NotNull(message = "Los cupos son obligatorios.")
    @Min(value = 1, message = "Debe haber al menos 1 cupo disponible.")
    private Integer cuposDisponibles;

    public CursoDTO() {}

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public Integer getCuposDisponibles() { return cuposDisponibles; }
    public void setCuposDisponibles(Integer cupos) {
        this.cuposDisponibles = cupos;
    }
}

```

3. Controlador REST (`CursoRestController.java`)

Cree la interfaz de controlador expuesta y documentada en la ruta:

`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica2/controller/CursoRestController.java`

```

package cr.ac.ucr.paraiso.ie.carnet.practica2.controller;

import cr.ac.ucr.paraiso.ie.carnet.practica2.business.MatriculaService;
import cr.ac.ucr.paraiso.ie.carnet.practica2.domain.Curso;
import cr.ac.ucr.paraiso.ie.carnet.practica2.dto.CursoDTO;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.Parameter;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

import java.util.ArrayList;
import java.util.List;

```

```

@RestController
@RequestMapping("/api/cursos")
@Tag(name = "Cursos", description = "Endpoints para la gestión de cursos académicos")
public class CursoRestController {

    private final MatriculaService matriculaService;

    // Inyección de dependencia por constructor
    public CursoRestController(MatriculaService matriculaService) {
        this.matriculaService = matriculaService;
    }

    @Operation(
        summary = "Obtener un curso por su identificador",
        description = "Retorna un curso específico si existe en base de datos."
    )
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Curso encontrado"),
        @ApiResponse(responseCode = "404", description = "Curso no encontrado")
    })
    @GetMapping("/{id}")
    public ResponseEntity<Curso> obtenerPorId(
        @Parameter(description = "ID único del curso", example = "1")
        @PathVariable Integer id) {

        // Simulación de búsqueda para el ejemplo conceptual
        if (id.equals(1)) {
            Curso cursoMock = new Curso("Desarrollo IV", 30);
            cursoMock.setId(1);
            return ResponseEntity.ok(cursoMock);
        }

        return ResponseEntity.notFound().build();
    }

    @Operation(
        summary = "Registrar un nuevo curso académico",
        description = "Guarda un curso en la base de datos tras validar campos."
    )
    @ApiResponses({
        @ApiResponse(responseCode = "201", description = "Curso creado"),
        @ApiResponse(responseCode = "400", description = "Datos de entrada inválidos")
    })
    @PostMapping
    public ResponseEntity<Curso> crear(
        @Validated @RequestBody CursoDTO dto) {

```

```

    // Mapeo e invocación simulada del servicio
    Curso nuevoCurso = new Curso(dto.getNombre(), dto.getCuposDisponibles());
    nuevoCurso.setId(10); // ID auto-generado simulado

    return ResponseEntity.status(HttpStatus.CREATED).body(nuevoCurso);
}
}

```

Explicación del Código Línea por Línea

- **CursoRestController.java (Línea 21):** `@RestController` registra la clase en el contexto de Spring Boot como un controlador de servicios web REST, asegurando que las respuestas de los métodos se escriban directo como JSON.
- **CursoRestController.java (Línea 23):** `@Tag(name = "Cursos", ...)` es una anotación de OpenAPI que agrupa visualmente los endpoints de este controlador bajo una pestaña con título descriptivo en la interfaz de Swagger.
- **CursoRestController.java (Líneas 33-36):** `@Operation` y `@ApiResponse` definen metadatos de documentación, indicando explícitamente el resumen de la operación y la lista de códigos de respuesta HTTP (200 y 404) que el cliente debe esperar.
- **CursoRestController.java (Líneas 40-41):** `@PathVariable` describe el argumento `@PathVariable` directamente en el panel de Swagger, permitiendo a los desarrolladores externos comprender qué tipo de dato pasar en la URL del navegador.
- **CursoDTO.java (Líneas 9-16):** La anotación `@Schema` documenta las propiedades de transferencia, asociándoles descripciones claras y ejemplos prácticos para autogenerar la documentación de la interfaz de Swagger.

Trampas Comunes

Retornar Siempre Código de Estado 200 OK:

Uno de los errores más graves es retornar excepciones u errores con un código HTTP 200 ok adjuntando el mensaje de error en el JSON. Esto rompe la semántica del protocolo HTTP e impide que los frameworks cliente (como Angular o Axios) capturen los errores de manera

adecuada a través de interceptores de red.

- *Solución:* Utilice clases de utilidad como `ResponseEntity` para retornar estados explícitos (400, 404, 500).

Acoplamiento de Entidades de Base de Datos en Endpoints:

Retornar directamente entidades JPA (ej. `@ManyToMany`) puede generar bucles de serialización infinita por referencias circulares (*Infinite Recursion Exception*).

- *Solución:* Mapee siempre las entidades a objetos DTO limpios antes de enviarlos en la respuesta HTTP.

Ignorar la Validación en la Entrada de Datos:

Omitir la anotación `@Validated` o `@Valid` al recibir objetos `@RequestBody` permite que entren datos nulos o vacíos en la capa de negocio, provocando errores en cascada.

- *Solución:* Decore los parámetros de entrada del método con `@Validated` para asegurar la interceptación automática de errores.

Pregunta de Reflexión

Si desarrollamos una API RESTful sin estado (*stateless*), el servidor no mantiene información sobre qué clientes están conectados en memoria RAM.

- ¿De qué manera esta propiedad del diseño REST mejora la escalabilidad horizontal de una arquitectura de microservicios distribuidos en la nube frente a un modelo clásico basado en sesiones de servidor?